

Security Findings — BugBounty Platform

🔴 Critical

1. Rate Limiting Bypass via `X-Forwarded-For` Header
2. Hardcoded `SECRET_KEY` in `settings.py`

🟡 High

3. Account Lockout Denial-of-Service
4. Upload Endpoint Crashes with HTTP 500 When ClamAV Is Unavailable
5. Stored XSS — No Input Sanitization on Report Fields

🟡 Medium

6. reCAPTCHA Effectively Disabled
7. Account Lockout Password Oracle
8. Server Version Disclosure

🔵 Low / Configuration

9. `Secure` Cookie Flag Defaults to `False`
10. Reporter User ID Leaked in API Responses
11. `security.txt` References a Nonexistent Domain

✅ What Works Correctly

Security Findings — BugBounty Platform

Live pentest run against the Docker stack on 2026-05-26. All findings were confirmed with real HTTP traffic against a running instance.

🔴 Critical

1. Rate Limiting Bypass via `X-Forwarded-For` Header

Confirmed: Every throttle class (login, register, submission) is keyed on the client IP address. Because `NUM_PROXIES` is not set in the DRF config, Django trusts any `X-Forwarded-For` value the client sends. An attacker can rotate IPs in that header and bypass all rate limits indefinitely.

Proof:

```
# After exhausting real IP limit (all 429):
curl -X POST /api/token/ -H "X-Forwarded-For: 10.20.30.1" -d '...' → 401 (not throttled)
curl -X POST /api/token/ -H "X-Forwarded-For: 10.20.30.2" -d '...' → 401 (not throttled)
...8 requests, all bypass the throttle
```

Impact: Full brute-force of any account’s password; automated mass registration; unlimited report submissions.

Fix — `Backend/Backend/settings.py` :

```
REST_FRAMEWORK = {
    ...
    'NUM_PROXIES': 0,    # Trust only the real TCP peer IP; ignore X-Forwarded-For
    ...
}
```

If deployed behind a real load balancer, set `NUM_PROXIES` to the exact number of trusted proxies instead of 0.

2. Hardcoded `SECRET_KEY` in `settings.py`

File: `Backend/Backend/settings.py` line 24

The Django secret key `django-insecure-@j0md9c!fn2v3r^ro2qis1y_x-gl0-*q#+bpm^buop*exz@x16` is hardcoded. This key is used as the HMAC secret for signing JWT tokens. Anyone who obtains the source code can forge valid access tokens for any user ID without needing credentials.

Proof: A startup `RuntimeWarning` fires on every container start confirming the insecure key is active:

```
RuntimeWarning: DJANGO_SECRET_KEY is using the insecure default – set it in production!
```

Impact: Full authentication bypass — forge a JWT for `user_id=1` (admin) and have unrestricted API access.

Fix:

```
# Generate a key
python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"
```

Set it as `DJANGO_SECRET_KEY` environment variable. Never commit the value. Add to `docker-compose.yml` :

```
environment:
  - DJANGO_SECRET_KEY=${DJANGO_SECRET_KEY}    # required, no default
```

High

3. Account Lockout Denial-of-Service

Confirmed: The `AccountLockout` model locks any account after ~5 failed password attempts for 30 minutes. Lockouts are per-username (not per-IP). Combined with finding #1, an attacker who knows a victim’s username can permanently lock them out by sending 5–6 wrong-password requests per 30-minute window — forever.

Proof:

```
# Loop 6 wrong-password requests for victim1 with rotating XFF headers
→ "Account is temporarily locked due to too many failed attempts. Please try again in 30 minute(s)."
```

Username are discoverable: report responses include `"reporter": 4` (user ID), and usernames appear in comment author fields and session endpoints.

Impact: Any known user can be locked out indefinitely, preventing them from submitting reports or accessing the platform.

Fix options: - Add per-IP lockout counter alongside per-account (so the attacker, not the victim, gets locked) - Add CAPTCHA to the login form - Notify the user by email when their account is locked so they can react - Use progressive delay (1s, 2s, 4s…) instead of hard lockout

4. Upload Endpoint Crashes with HTTP 500 When ClamAV Is Unavailable

File: `Backend/reports/views.py` , `upload_attachment` action

The `ClamdNetworkSocket(host=..., port=...)` constructor is called **outside** the `try/except` block. If ClamAV is unreachable, the constructor raises `pyclamd.ConnectionError` which is not caught, resulting in an unhandled exception and HTTP 500.

Affected code:

```
# VULNERABLE – constructor outside try/except
cd = ClamdNetworkSocket(host=os.getenv("CLAMAV_HOST", "clamav"),
                        port=int(os.getenv("CLAMAV_PORT", "3310")))

try:
    scan_result = cd.scan_file(tmp_path) # only this is guarded
except Exception:
    scan_result = None
```

The ClamAV container (`clamav/clamav:1.4`) is also currently in a restart loop — the upload endpoint is non-functional in the current deployment.

Fix:

```
try:
    cd = ClamdNetworkSocket(host=os.getenv("CLAMAV_HOST", "clamav"),
                           port=int(os.getenv("CLAMAV_PORT", "3310")))
    scan_result = cd.scan_file(tmp_path)
except Exception:
    scan_result = None

if not scan_result:
    os.unlink(tmp_path)
    return Response({"detail": "Upload blocked: virus scan unavailable."}, status=400)
```

Also fix the ClamAV container — the `mkodockx/docker-clamav:alpine` image referenced in the project notes does not match `clamav/clamav:1.4` which is what's actually running and crashing.

5. Stored XSS — No Input Sanitization on Report Fields

Confirmed: HTML/JS payloads are accepted and stored verbatim in `title`, `description`, and other text fields. The value is returned raw in API responses.

Proof:

```
POST /api/reports/ {"title": "<script>alert(document.cookie)</script>", ...}
→ 201 Created
GET /api/reports/2/ → "title": "<script>alert(document.cookie)</script>"
```

The deployed CSP (`script-src 'self'`) blocks inline script execution in the browser for the React SPA, and no `dangerouslySetInnerHTML` was found in the frontend. However: - The Django admin panel renders these fields as HTML — the payload executes there - Any future email templates that include report content would execute the payload - A CSP bypass or future feature could re-expose this

Fix: Sanitize at write time using `bleach` :

```
import bleach
title = bleach.clean(request.data.get('title', ''), tags=[], strip=True)
```

Or enforce it in the serializer's `validate_<field>` methods.

● Medium

6. reCAPTCHA Effectively Disabled

File: `Backend/docker-compose.yml` and `Backend/reports/captcha.py`

`RECAPTCHA_SECRET_KEY` defaults to an empty string in `docker-compose.yml` (`${RECAPTCHA_SECRET_KEY:-}`). When the secret is empty, `verify_recaptcha()` is an explicit no-op (returns `None` silently). This disables CAPTCHA protection on registration, file upload, and resend-verification — enabling bot-driven automation of all three.

Proof:

```
verify_recaptcha(None, None) # returns None – no exception raised
```

Fix: Make the key mandatory — remove the `:-` fallback and fail at startup if it's unset:

```
# docker-compose.yml
environment:
  - RECAPTCHA_SECRET_KEY=${RECAPTCHA_SECRET_KEY} # no default – must be set
```

7. Account Lockout Password Oracle

File: `Backend/users/views.py`, `EmailVerificationTokenSerializer.validate()` (lines 120–166)

The lockout check happens **after** the password check. This creates an observable difference on locked accounts: - **Correct password on locked account** → "Account is temporarily locked..." - **Wrong password on locked account** → "No active account found with the given credentials"

An attacker can confirm a found password is correct even while the account is locked, without triggering any alerts.

Fix — reorder the checks:

```
def validate(self, attrs):
    ...
    # 1. Check lockout FIRST before touching password
    lockout = AccountLockout.get_or_create_for_user(user)
    if lockout.is_locked():
        raise serializers.ValidationError("Account is temporarily locked...")

    # 2. Then check password
    if not user.check_password(password):
        lockout.record_failure()
        raise AuthenticationFailed('No active account found...')
```

8. Server Version Disclosure

Every response includes:

```
Server: WSGIServer/0.2 CPython/3.12.13
```

This reveals the exact Python version, which helps attackers look up known CVEs for that runtime.

Fix: Run behind nginx or gunicorn in production — both suppress the `Server` header or replace it with a non-identifying value.

Low / Configuration

9. Secure Cookie Flag Defaults to False

`JWT_AUTH_COOKIE_SECURE` reads from `DJANGO_SECURE_COOKIES` env var, which defaults to `False`. The `bb_access` and `bb_refresh` cookies will be sent over plain HTTP unless this is explicitly set. Easy to forget at deploy time.

Fix: Set `DJANGO_SECURE_COOKIES=True` and `DJANGO_SECURE_SSL=True` in production. Add a startup assertion that checks `DEBUG=False` implies `DJANGO_SECURE_COOKIES=True`.

10. Reporter User ID Leaked in API Responses

Report responses include `"reporter": 4` (internal database primary key). This can be used to correlate reports to specific users and feed the account-lockout DoS if usernames are also discoverable.

Fix: Replace the integer ID with the username or a public handle in the serializer's `reporter` field, or omit it for non-privileged callers.

11. security.txt References a Nonexistent Domain

`/.well-known/security.txt` returns contact info for `bugbounty.example.com`. This must be updated to a real contact address before the platform is reachable to external researchers.

What Works Correctly

Area	Status
IDOR on reports (queryset filters by <code>reporter=request.user</code>)	✔ Blocked
JWT algorithm confusion (<code>alg: none</code>)	✔ Rejected
JWT token tampering (modified payload, original signature)	✔ Rejected
Session revocation scoped to <code>user=request.user</code> (no IDOR)	✔ Correct

Area	Status
Password reset does not reveal email existence	✔ Correct
Double-extension upload bypass (<code>.jpg.php</code>)	✔ Blocked
HttpOnly + SameSite=Strict on JWT cookies	✔ Set
CSRF enforcement on cookie-authenticated requests	✔ Active
MFA required for Triager/Admin roles	✔ Enforced
Mass assignment (groups/role via registration)	✔ Blocked for <code>Admin</code> ; <code>company</code> role accepted by design
Content-Security-Policy headers	✔ Present
X-Frame-Options, X-Content-Type-Options	✔ Set
SQL injection via search/filter params	✔ ORM prevents it
Unauthenticated report access	✔ Blocked